Computer Architecture Lab WriteUp 2

4 March, 2026

1 What Even is a CPU and an ALU?

Let's start from absolute zero. Imagine you have a very fast but completely dumb calculator. It doesn't know what to do unless you give it super specific instructions.

A **CPU** (Central Processing Unit) is basically the brain of the computer. It reads instructions from a memory block, figures out what they mean, and then executes them. But the CPU itself is made of smaller parts. The part that actually does the heavy lifting—the real math and logic stuff—is the **ALU** (Arithmetic Logic Unit).

If the CPU is a restaurant kitchen, the ALU is the chef who actually chops the vegetables and cooks the meat. The ALU takes two numbers, does a specific math or logic operation on them (like adding them, or comparing them), and spits out the result.

In this document, we are going to look at the exact code that builds this chef, and then the code that tests if the chef actually knows how to cook without burning the kitchen down.

2 Step 1: The ALU Code (`alu.sv`)

The file `alu.sv` is the actual hardware design. When we write SystemVerilog, we aren't writing normal software that runs on a computer. We are actually describing digital wires and logic gates that will be printed onto a silicon chip.

2.1 The Inputs and Outputs (The Header)

Every module in hardware is like a physical box with wires coming in and wires going out.

```
module alu (
    input  logic [31:0] operand_a,
    input  logic [31:0] operand_b,
    input  logic [3:0]  alu_operation,

    output logic [31:0] result,
    output logic        zero
);
```

Here is the breakdown of the box:

- `operand_a` and `operand_b`: These are the two 32-bit numbers we want to do math on. Think of '[31:0]' as a bundle of 32 physical wires.
- `alu_operation`: This is a 4-bit wire bundle coming from the boss (the controller). It tells the ALU *which* operation to perform. Because it has 4 bits, it can hold up to 16 different commands ($2^4 = 16$).

- `result`: The final 32-bit answer that the ALU spits out.
- `zero`: A single tiny wire that turns ON (becomes 1) if the final result is exactly zero. We use this later to help the CPU make decisions (like "if A - B equals 0, it means they are equal, so jump to another part of the code").

2.2 The Brain of the ALU

```
always_comb begin
    unique case (alu_operation)
        4'b0000: result = operand_a & operand_b;
    // AND
        4'b0010: result = operand_a + operand_b;
    // ADD
        // ... other operations ...
        default: result = 32'd0;
    endcase
end
```

- `always_comb`: This means "always combinational". It tells the hardware that this isn't a memory block. It doesn't wait for a clock tick. The second the inputs change, the electricity flows through the gates and the output changes instantly.
- `unique case`: This is a switchboard. It looks at the 4-bit command and routes the data to the correct math circuit. `4'b0000` means "a 4-bit binary number with the value 0000".

2.3 Some Weird Operations Explained

Most of the operations are basic math like `+` or `-`. But some look strange:

- **Shifts (`<<`, `>>`, `>>>`)**: These move all the bits left or right. If you have the binary number for 2 (`0010`) and shift it left by 1, it becomes 4 (`0100`). The `>>>` is a "signed" shift, meaning if the number was a negative number, it keeps the minus sign attached when moving bits around.
- **SLT (Set Less Than) `<`**: This compares the two numbers. If A is less than B, the result is 1. If not, the result is 0. We use `$signed()` to tell the hardware, "Hey, treat these as positive and negative numbers, not just raw data."

3 Step 2: Testing the ALU (`alu_tb.sv`)

You can't just write hardware code and pray it works. Once a chip is printed, you can't fix a bug. So, we write a "Testbench" (TB).

A testbench is basically a fake software world. We create a fake motherboard, plug our ALU module into it, and then blast it with thousands of random numbers to see if it makes a mistake.

3.1 Plugging It In

```
alu dut (
    .operand_a(operand_a),
    .operand_b(operand_b),
    .alu_operation(alu_operation),
    .result(result),
    .zero(zero)
);
```

"DUT" stands for Design Under Test. This block of code is literally us taking the inputs/outputs of our real ALU box and wiring them up to the fake wires in our testbench.

3.2 The Brute Force Attack

```
for (int i = 0; i < 10000; i++) begin
    operand_a     = $urandom();
    operand_b     = $urandom();
    alu_operation = $urandom_range(0, 9);
    // ...
```

We don't want to type out tests manually. We use a `for` loop to run 10,000 tests. The `$urandom()` function just generates completely random 32-bit garbage numbers, and `$urandom_range(0, 9)` picks a random math operation from 0 to 9. We are basically giving our chef completely random ingredients and telling him to cook random meals at lightning speed.

3.3 The Checker

How does the testbench know if the ALU's answer is correct? The testbench actually acts like a second, perfect software calculator.

```
if (result !== expected_result || zero !== expected_zero) begin
    $display("----> FAIL");
    $fatal;
end
```

The testbench computes what the answer *should* be (we call this `expected_result`). Then, it waits 1 nanosecond (`#1;`), looks at the real hardware's output (`result`), and compares them.

If they do not match exactly (`!==`), the code screams "FAIL", prints out the exact numbers that broke it, and uses `$fatal` to completely crash the simulation. If it survives all 10,000 random tests without hitting `$fatal`, we print a massive success message, and we know our ALU design is bulletproof.

4 Step 3: The Controller (The Restaurant Manager)

4.1 What is a Controller?

If the ALU is the chef cooking the food, the **Controller** is the Restaurant Manager. The manager stands at the front, takes the customer's order (the 32-bit Instruction), and then starts flipping switches and yelling at everyone in the kitchen so the order gets made correctly.

”Chef, chop this!” (ALU control). ”Waiter, write this down in the ledger!” (Register Write). ”Open the freezer!” (Memory Read).

Without the Controller, the CPU is just a bunch of disconnected parts staring at each other.

4.2 Splitting the Job: Main Boss and Micro-Manager

Reading a 32-bit instruction is complicated. Instead of having one guy try to do everything, we split the manager’s job into two smaller jobs:

- **Main Decoder (The General Manager):** This guy only looks at the first 7 bits of the order (the `opcode`). He figures out the general vibe. Is this a math problem? Are we loading data from memory? He sets up the main switches and passes a simple sticky note (`alu_op`) to his assistant.
- **ALU Decoder (The Kitchen Micro-Manager):** This guy reads the sticky note from the boss, and then looks at the tiny details of the order (`funct3` and `funct7`). He then turns around and tells the chef (the ALU) *exactly* which 4-bit math operation to do.

4.3 The Manager’s Megaphone (Control Signals)

Here are the actual switches the Main Boss flips:

- `reg_write`: ”Save this result in the notebook!” (Writes to the Register File).
- `alu_src`: ”Chef! Do you want the ingredient from the fridge (Register), or the exact number I’m handing you right now (Immediate)?”
- `mem_write`: ”Overwrite the data in the main safe!” (Writes to Data Memory).
- `branch`: ”Wait, the customer changed their mind, skip to page 5 of the recipe book!”

5 Step 4: The Controller Code

5.1 The Main Boss (`main_decoder.sv`)

The Main Boss uses a `case` statement to look at the `opcode`. Depending on the family of the instruction, he flips the right wires to 1 or 0.

```
module main_decoder (
    input  logic [6:0] opcode,
    output logic       branch,
    output logic       result_src,
    output logic       mem_write,
    output logic       alu_src,
    output logic [1:0] imm_src,
    output logic       reg_write,
    output logic [1:0] alu_op
);

```

```
always_comb begin
    // First, turn everything OFF by default so we don't accidentally cook the wrong thing
    branch     = 1'b0;
    result_src = 1'b0;
    mem_write  = 1'b0;
    alu_src    = 1'b0;
    imm_src    = 2'b00;
    reg_write  = 1'b0;
    alu_op     = 2'b00;

    unique case (opcode)
        // Load (lw) - "Go fetch data from the safe"
        7'b0000011: begin
            alu_src    = 1'b1;
            result_src = 1'b1;
            reg_write  = 1'b1;
            imm_src    = 2'b00;
            alu_op     = 2'b00;
        end
        // Store (sw) - "Put this data into the safe"
        7'b0100011: begin
            alu_src   = 1'b1;
            mem_write = 1'b1;
            imm_src   = 2'b01;
            alu_op    = 2'b00;
        end
        // R-Type (add, sub) - "Hey Chef, do some math!"
        7'b0110011: begin
            reg_write = 1'b1;
            alu_op    = 2'b10;
        end
        // Branch (beq) - "Compare these and maybe jump!"
        7'b1100011: begin
            branch  = 1'b1;
            imm_src = 2'b10;
            alu_op  = 2'b01;
        end
        default: ; // If it's garbage, do nothing.
    endcase
end

endmodule
```

5.2 The Micro-Manager (`alu_decoder.sv`)

This is where we actually output the exact 4-bit `alu_operation` codes we designed in Step 1.

```
module alu_decoder (
    input  logic       op_5,   // instruction[30] (tells us if it's ADD or SUB)
    input  logic [2:0] funct3,
    input  logic [1:0] alu_op, // The sticky note from the Boss
    output logic [3:0] alu_operation
);

```

```
always_comb begin
    unique case (alu_op)
        2'b00: alu_operation = 4'b0010; // Load/Store always uses ADD
        2'b01: alu_operation = 4'b0110; // Branch always uses SUB (to compare)

        2'b10: begin // R-type math operations
            unique case (funct3)
                3'b000: alu_operation = op_5 ? 4'b0110 : 4'b0010; // SUB or ADD
                3'b010: alu_operation = 4'b1000; // SLT (Set Less Than)
                3'b110: alu_operation = 4'b0001; // OR
                3'b111: alu_operation = 4'b0000; // AND
                default: alu_operation = 4'b0000;
            endcase
        end

        default: alu_operation = 4'b0000;
    endcase
end

endmodule
```

6 Step 5: Testing the Managers (`controller_tb.sv`)

For the ALU, we used a "Brute Force Attack" with completely random garbage numbers because math has infinite combinations.

But you can't give the Controller (the manager) completely random garbage instructions. If a CPU reads an instruction that doesn't exist, it crashes. So instead of a pre-written pop quiz, our testbench simulates a chaotic **Friday Night Dinner Rush** using a method called **Constrained Random Testing**.

We throw 5,000 random orders at the manager, but we *only* order items that actually exist on the menu.

6.1 The Dinner Rush Code

Here is how we set up the rush:

1. **The Menu:** We create arrays (`valid_opcodes`, `valid_func3`, etc.) that only contain legal RISC-V instructions.

2. **The Orders:** We loop 5,000 times, blindly throwing a dart at our menu to generate a random—but completely legal—instruction.

3. **The Phantom Manager:** The testbench acts as a second, perfect manager to calculate what the output *should* be (`expected_alu_op`).

4. **The Health Inspector:** If the real hardware manager messes up the order (`alu_operation !== expected_alu_op`), we scream "FAIL", print the exact order that caused the choke, and shut down the simulation (`$fatal`).

```
`timescale 1ns/1ps
`include "../rtl/opcode.vh"

module controllers_tb;

    // inputs
    logic [6:0] opcode;
    logic [2:0] func3;
    logic [6:0] func7;

    // connection between controllers
    logic [1:0] alu_op;

    // final output
    logic [3:0] alu_operation;
    logic [3:0] expected_alu_op;

    // instantiate controllers
    main_controller main_ctrl (.opcode(opcode), .alu_op(alu_op));
    alu_controller alu_ctrl (.alu_op(alu_op), .func3(func3), .func7(func7), .alu_operation(alu_operation));

    // lists of only the supported stuff (The Menu)
    logic [6:0] valid_opcodes [5] = '{ `OPC_LOAD, `OPC_STORE, `OPC_BRANCH, `OPC_ARI_RTYPE, `OPC_ARI_ITYPE };
    logic [2:0] valid_func3 [8]   = '{ `FNC_ADD_SUB, `FNC_SLL, `FNC_SLT, `FNC_SLTU, `FNC_XOR, `FNC_SRL_SRA, `FNC_OR, `FNC_AND };
    logic [6:0] valid_func7 [2]   = '{ `FNC7_0, `FNC7_1 };

    initial begin
        $dumpfile("controllers.vcd");
        $dumpvars(0, controllers_tb);

        $display("=============================================");
        $display("starting constrained random testing...");
        $display("=============================================");

        // The 5000 Order Rush
        for (int i = 0; i < 5000; i++) begin
            // pick a random index from the valid arrays
            opcode = valid_opcodes[$urandom_range(0, 4)];
            func3  = valid_func3[$urandom_range(0, 7)];
            func7  = valid_func7[$urandom_range(0, 1)];

            #1;

            // The Phantom Manager calculates the perfect answer
            if (opcode == `OPC_LOAD || opcode == `OPC_STORE)
                expected_alu_op = 4'b0010;
            else if (opcode == `OPC_BRANCH)
                expected_alu_op = 4'b0110;
            else if (opcode == `OPC_ARI_RTYPE || opcode == `OPC_ARI_ITYPE) begin
                case (func3)
                    `FNC_ADD_SUB: expected_alu_op = (opcode == `OPC_ARI_RTYPE && func7 == `FNC7_1) ? 4'b0110 : 4'b0010;
                    `FNC_SLL: expected_alu_op = 4'b0100;
                    `FNC_SLT: expected_alu_op = 4'b1000;
```

```
                    `FNC_SLTU: expected_alu_op = 4'b1001;
                    `FNC_XOR: expected_alu_op = 4'b0011;
                    `FNC_SRL_SRA: expected_alu_op = (func7 == `FNC7_1) ? 4'b0111 : 4'b0101;
                    `FNC_OR:  expected_alu_op = 4'b0001;
                    `FNC_AND: expected_alu_op = 4'b0000;
                    default:  expected_alu_op = 4'b0000;
                endcase
            end else begin
                expected_alu_op = 4'b0010;
            end

            // Status update to the terminal
            if (i % 250 == 0) begin
                $display("test %0d | opcode: %b | func3: %b | func7: %b | expected: %b | got: %b", i, opcode, func3, func7, expected_alu_op, alu_operation);
            end

            // The Health Inspector Check
            if (alu_operation !== expected_alu_op) begin
                $display("----> FAIL at test %0d", i);
                $display("opcode: %b | func3: %b | func7: %b", opcode, func3, func7);
                $display("expected: %b | got: %b", expected_alu_op, alu_operation);
                $fatal;
            end
        end

        $display("==============================================");
        $display("all 5000 constrained random tests PASSED.");
        $display("==============================================");

        $finish;
    end

endmodule
```